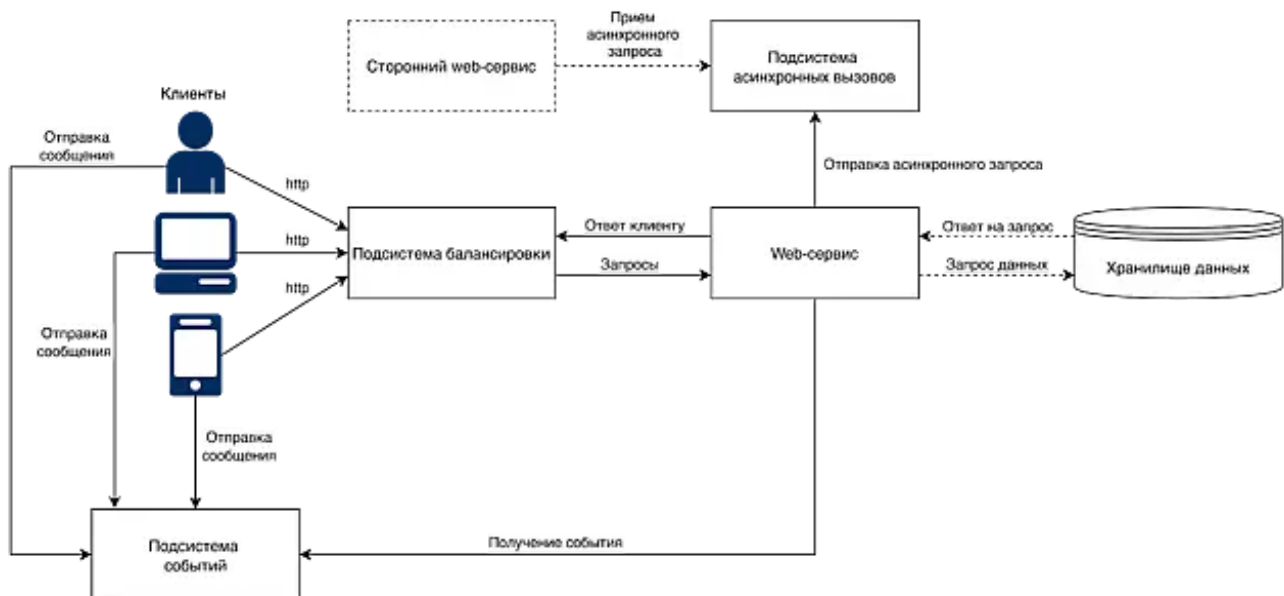


Узлы облака

Участок: Управление облаком (cloud-ctrl)

Основными узлами облака являются:

- **Клиент** — инициатор запроса. В качестве клиентов могут выступать браузер, скрипт, мобильное приложение и т.д., способные генерировать запросы к web-сервисам по http-протоколу.
- **Подсистема балансировки** — прозрачный прокси-сервер, распределяющий нагрузку на узлы web-сервисов. Нагрузка обычно распределяется по алгоритму [round-robin](#), но может быть заменена на другой.
- **Web-сервис** — облачный ресурс, который обладает собственным URL, по которому отсылаются запросы на исполнение. Web-сервис исполняет запросы как внешних, так и внутренних клиентов.
- **Подсистема асинхронных вызовов** — обеспечивает асинхронную обработку запросов между сервисами.
- **Подсистема событий** — организует взаимодействие между клиентом и web-сервисом через брокер сообщений, а также отправку клиентских и серверных событий.



Подсистемы облака реализованы на базе **кластеров** - набора программ, которые объединяются по общему назначению и используются как единое целое.

Балансировщик

Балансировщики осуществляют проверку данных пользователя и обеспечивают [маршрутизацию запросов](#) к web-сервисам с помощью вычисления конкретного узла сервиса.

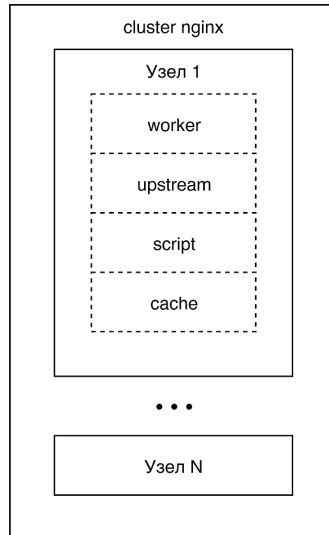
Балансировщики делятся на три типа:

- **внешние** - принимают запросы от конечных пользователей (браузеров, мобильных приложений) и распределяют их на внутренние сервисы.
- **внутриоблачные** - используются для взаимодействия сервисов между собой внутри облака.
- **межоблачные** - используются для взаимодействия web-сервисов различных облаков.

Пример реализации http-балансировщика — кластер `nginx`. В зависимости от настройки и размещения, он может быть как внешним, так и внутриоблачным балансировщиком. Сам кластер состоит из узлов, которые включают в себя компоненты:

- **worker** — процесс, принимающий запросы от клиента. Он выполняет всю основную работу.
- **cache** — кэш. Хранит некоторую информацию для быстрого доступа.
- **script** — lua-скрипт, в котором закодированы алгоритмы маршрутизации и другие полезности.
- **upstream** — список, элементами которого являются описания web-сервисов, включенных в сайт. Каждый элемент содержит:

- системное имя сервиса
- идентификатор группы (group-id)
- url-путь web-сервиса
- тип элемента
- список узлов web-сервиса, состоящий из пар хост:порт
 - хост — имя сервера, где расположен узел web-сервиса или tenant-группы.
 - порт — порт, который этот узел слушает.



Web-сервисы

Web-сервис - это [сервис](#), обладающий URL с протоколом передачи данных ([http](#) - для синхронных вызовов, [amqp](#) - для асинхронных).

Web-сервисы облака делятся на три типа:

- [простые сервисы](#)
- [multiversion-сервисы](#)
- [multitenancy-сервисы](#)

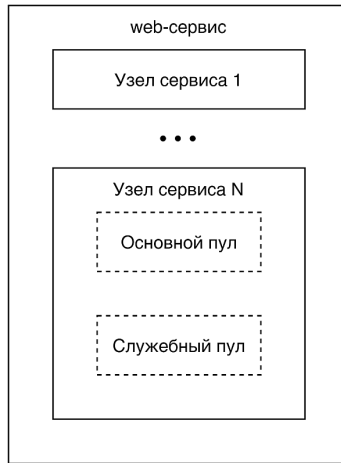
Каждый web-сервис состоит из **узлов** — физических экземпляров развернутого сервиса, обеспечивающий полную или частичную функциональность.

Простые сервисы

Простые web-сервисы состоят из нескольких отдельных равноправных узлов.

Каждый узел сервиса может иметь два пула очередей запросов на исполнение:

- основной - предназначен для исполнения более приоритетных запросов; например, запросы с БЛ, запросы с интерфейса.
- служебный - предназначен для исполнения "тяжеловесных" запросов, зачастую асинхронных.



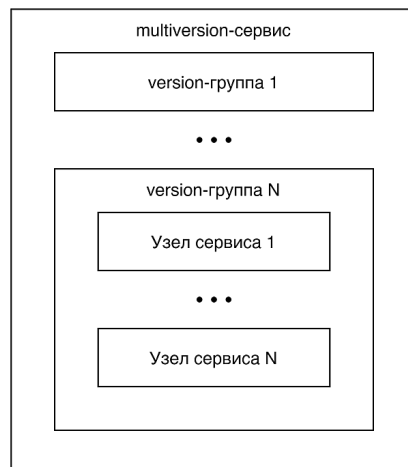
По умолчанию запросы обрабатываются основным пулом. Если требуется, чтобы запрос был обработан служебным пулом, то для этого в параметрах http-запроса нужно передать строку: ?srv=1.

Multiversion-сервисы

Multiversion-сервис — это web-сервис, имеющий несколько реализаций (**version-групп**). Все version-группы одного сервиса имеют одинаковый url-адрес, но могут друг от друга отличаться набором модулей и базами данных. Каждая version-группа состоит из отдельных равноправных полнофункциональных узлов и имеет свой уникальный номер (**group-id**).

Version-группы делятся на два вида:

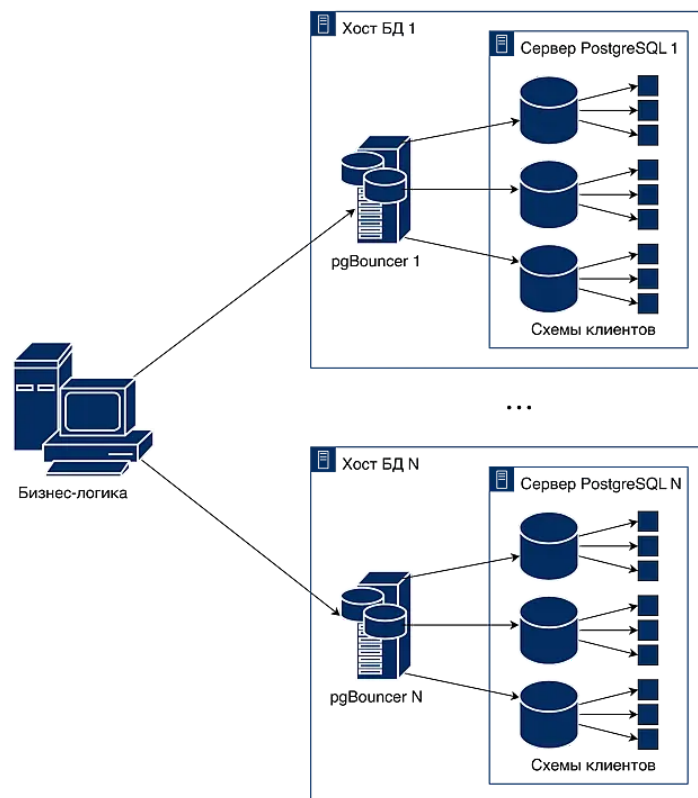
- **реализация по умолчанию** - на такую группу направляются запросы, у которых нет явно заданной версии. Реализация по умолчанию может отсутствовать, в этом случае запросы на нее будут подавлены с помощью version-контекстов.
- **реализация под контекст** - запросы направляются в конкретную version-группу строго по правилу из [контекста](#).



Multitenancy-сервисы

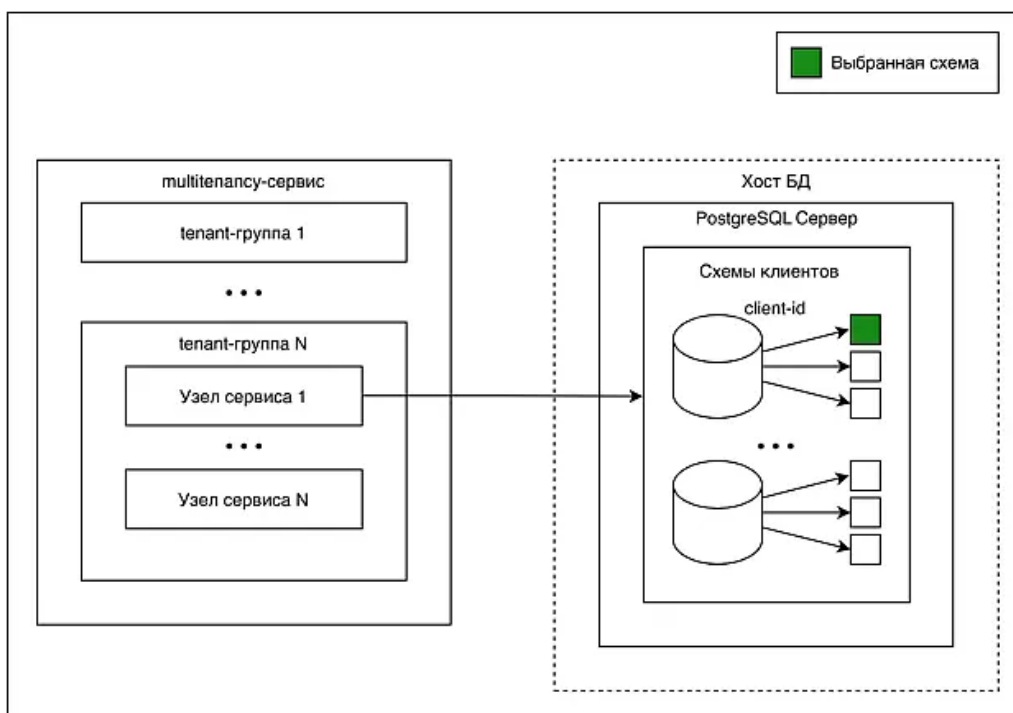
Multitenancy-сервис является частным случаем multiversion, где версией является не отдельный экземпляр сервиса, а изолированная среда для тенанта (**tenant**) - клиента, использующего ресурсы облака независимо от других клиентов.

Персональные данные изолируются с помощью **схем** — отдельных наборов таблиц для каждого клиента.



Однако выделять ресурсы облака под каждого клиента очень дорого, а предоставлять их всем клиентам неудобно, т.к. это создает точки отказа. Поэтому tenants объединяются в **tenant-группы**, каждая из которых состоит из отдельных узлов, имеет уникальный номер **group-id** и обслуживает несколько баз данных. Url-адрес для каждой tenant-группы одного сервиса одинаковый. Каждая база данных внутри группы имеет свой уникальный номер **dbid**.

Сопоставление схемы и клиента выполняется по **client-id** — идентификатору клиента в облаке.



Подсистема асинхронных вызовов

Асинхронные вызовы применяют для того, чтобы система могла эффективно использовать ресурсы при выполнении задач, результат которых не требуется для немедленного продолжения работы.

Для обработки асинхронных вызовов используется брокер сообщений. В нашей системе брокер реализован с помощью [RabbitMQ](#). Альтернативная реализация — [request-broker](#). В отличие от RabbitMQ, он позволяет отследить текущее состояние любого запроса по его идентификатору.

Клиент (сервис, отправляющий запрос) определяет информацию о получателе по данным, переданным в вызов (системное имя сервиса, идентификатор пользователя и т.п.) и сессии вычисляет обменник — место отправки запросов. Из обменника запросы попадают в [очереди](#), а из очередей их забирают узлы web-сервиса.

Очереди делятся на следующие типы:

- основная — в нее попадают вызовы, которые готовы к немедленной обработке;
- отложенная — в ней хранятся вызовы, которые должны быть выполнены в будущем. Web-сервис перенаправляет в эту очередь вызовы, которые не готов обработать сейчас. Например, при [обновлении "мастер-реплика"](#) асинхронный вызов модифицирующего метода перенаправляется в отложенную очередь.

Из обменника запрос передается только в основную очередь.



Когда web-сервис забирает запрос из очереди, он проводит проверку на актуальность, т.к. пока запрос находился в очереди, могло начаться обновление и клиент, которому адресован запрос, уже обслуживается новой версией бизнес-логики. В этом случае web-сервис повторно направляет запрос в нужную очередь, либо выполняет его.

Request-broker работает аналогично.

Подсистема событий

Событие - это оповещение о некотором действии или изменении в системе.

В зависимости от [сценария доставки](#) события делятся на два типа:

- [клиентские](#) - доставляются из облака до клиентов (браузера, плагина или мобильного приложения).

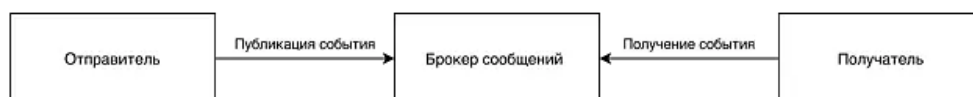
Пример использования: при получении нового требования из налоговой инспекции в браузер клиента передается соответствующее событие. Браузер обрабатывает событие и отображает сообщение в интерфейсе.

- [серверные](#) - используются для взаимодействия сервисов между собой.

Пример использования: сервис прав отправляет другим сервисам событие о том, что права пользователя изменились.

Основные компоненты подсистемы событий:

1. Отправитель (producer) - публикует событие, передавая сообщение в брокер. В качестве отправителя выступает web-сервис.
2. Брокер сообщений - организует доставку сообщения между отправителем и получателем. Передача сообщений выполняется по протоколу [AMQP](#).
3. Получатель (consumer) - получает и обрабатывает событие из очередей брокера. В качестве получатель выступает клиент или web-сервис, в зависимости от типа события. Для старта обработки получатель должен подписаться на очередь брокера.



В облаке подсистема событий реализована на RabbitMQ, аналогично [подсистеме асинхронных вызовов](#). Основное отличие - получатель события неизвестен, т.е. тот, кто передает событие не знает, кто получит событие и есть ли получатель вообще.

Отправитель передает сообщение в обменник, соответствующий событию. Обменник доставляет событие по очередям с помощью связывания данных (binding). На один обменник может быть подписано несколько очередей, которые в зависимости от типа события различаются:

- для клиентских событий - очереди пользователя,
- для серверных событий - очереди бизнес-логики.

Очередь бизнес-логики может сохраняться на диск (durable) или только в ОЗУ (transient). Очередь пользователей сохраняется только в ОЗУ.

